



**BSc (Hons) in Computer Science
Year 3**

Lab Sheets

SE3082 – Parallel Computing

Semster 1, 2026

Lab 05

IT24101022 – Mannapperuma.M.A.S.K.M.

Exercise 01

The `#pragma omp barrier` directive synchronizes all threads within the parallel region. Each thread must wait at the barrier until all other threads have reached the same point. Once all threads reach the barrier, they can continue executing the statements after the barrier.

```
wsl@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise01
16 directories, 15 files
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06$ cd ~/SE3082-2026-Sem02-Lab06/Exercise01
./exercise1
Thread # 6
Thread # 4
Thread # 3
Thread # 7
Thread # 2
Thread # 5
Thread # 0
Thread # 1
Outside Barrier # 1
Outside Barrier # 7
Outside Barrier # 5
Hi again from thread # 7
Outside Barrier # 6
Hi again from thread # 6
Outside Barrier # 4
Hi again from thread # 1
Hi again from thread # 4
Outside Barrier # 0
Hi again from thread # 5
Hi again from thread # 0
Outside Barrier # 3
Hi again from thread # 3
Outside Barrier # 2
Hi again from thread # 2
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise01$
```

Exercise 02

The `#pragma omp single` directive ensures that the code inside the single block is executed by only one thread among all threads in the parallel region. The other threads skip the single block and continue with the statements after it. Therefore, the messages “Inside Single” and “Exiting Single” are printed only once, while “Hi again” is printed by every thread.

```
wsl@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise02
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise01$ cd ~/SE3082-2026-Sem02-Lab06/Exercise02
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise02$ ls
exercise02.txt  exercise2.c  screenshots
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise02$ gcc -fopenmp exercise2.c -o exercise2
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise02$ ./exercise2
thread # 5
Inside Single # 5
Exiting Single
thread # 1
thread # 7
thread # 4
thread # 0
thread # 3
thread # 6
thread # 2
Hi again from thread # 4
Hi again from thread # 5
Hi again from thread # 0
Hi again from thread # 1
Hi again from thread # 7
Hi again from thread # 6
Hi again from thread # 2
Hi again from thread # 3
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise02$
```

Exercise 03

The `#pragma omp master` directive ensures that the code inside the master block is executed only by the master thread, which is normally thread 0. The other threads do not execute the master block and continue with the following statements. Unlike the `omp single` directive, the `omp master` directive does not create an implicit barrier at the end of the master block.

```
wsl@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise03
Hi again from thread # 3
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise03$ cd ~/SE3082-2026-Sem02-Lab06/Exercise03
ls
exercise03.txt  exercise3.c  screenshots
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise03$ gcc -fopenmp exercise3.c -o exercise3
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise03$ ./exercise3
thread # 4
thread # 3
Hi again from thread # 3
Hi again from thread # 4
thread # 1
thread # 6
Hi again from thread # 6
thread # 7
Hi again from thread # 7
Hi again from thread # 1
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 2
Hi again from thread # 2
thread # 5
Hi again from thread # 5
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise03$
```

Exercise 04

With `#pragma omp parallel for`, OpenMP parallelizes only the outer loop. Therefore, the five iterations of the `r` loop are distributed among the available threads, while each assigned thread executes its corresponding inner `k` loop.

When `collapse(2)` is used, the two nested loops are combined into a single iteration space. Therefore, the $5 \times 10 = 50$ combinations of `r` and `k` are distributed among the available threads. This provides a larger iteration space and allows the workload to be distributed more evenly among the threads.

```

ws1@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise04
Hi again from thread # 5
ws1@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise03$ cd ~/SE3082-2026-Sem02-Lab06/Exercise04
ls
exercise04.txt  exercise4.c  screenshots
ws1@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise04$ gcc -fopenmp exercise4.c -o exercise4
ws1@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise04$ ./exercise4
r = 3, k = 1, thread # = 2
r = 3, k = 2, thread # = 2
r = 3, k = 3, thread # = 2
r = 3, k = 4, thread # = 2
r = 3, k = 5, thread # = 2
r = 3, k = 6, thread # = 2
r = 5, k = 1, thread # = 4
r = 5, k = 2, thread # = 4
r = 5, k = 3, thread # = 4
r = 4, k = 1, thread # = 3
r = 4, k = 2, thread # = 3
r = 3, k = 7, thread # = 2
r = 3, k = 8, thread # = 2
r = 3, k = 9, thread # = 2
r = 3, k = 10, thread # = 2
r = 2, k = 1, thread # = 1
r = 2, k = 2, thread # = 1
r = 2, k = 3, thread # = 1
r = 2, k = 4, thread # = 1
r = 2, k = 5, thread # = 1
r = 2, k = 6, thread # = 1
r = 2, k = 7, thread # = 1
r = 2, k = 8, thread # = 1
r = 2, k = 9, thread # = 1
r = 2, k = 10, thread # = 1
r = 5, k = 4, thread # = 4
r = 5, k = 5, thread # = 4
r = 5, k = 6, thread # = 4
r = 5, k = 7, thread # = 4
r = 5, k = 8, thread # = 4
r = 5, k = 9, thread # = 4
r = 5, k = 10, thread # = 4
r = 4, k = 3, thread # = 3
r = 4, k = 4, thread # = 3
r = 4, k = 5, thread # = 3
r = 4, k = 6, thread # = 3
r = 4, k = 7, thread # = 3
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0

```

Exercise 05

The recursive Fibonacci function is parallelized using OpenMP tasks. The two recursive calls, $\text{fib}(n-1)$ and $\text{fib}(n-2)$, are independent and can therefore be created as separate tasks. OpenMP can execute these tasks concurrently using the available threads. The `taskwait` directive ensures that both recursive tasks are completed before their results are added together and returned.

```
Select wsl@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise05
r = 4, k = 8, thread # = 3
r = 4, k = 9, thread # = 3
r = 4, k = 10, thread # = 3
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise04$ cd ~/SE3082-2026-Sem02-Lab06/Exercise05
ls
exercise05.txt  exercise5.c  screenshots
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise05$ gcc -fopenmp exercise5.c -o exercise5
./exercise5
Fibonacci(10) = 55
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise05$
```

Exercise 06

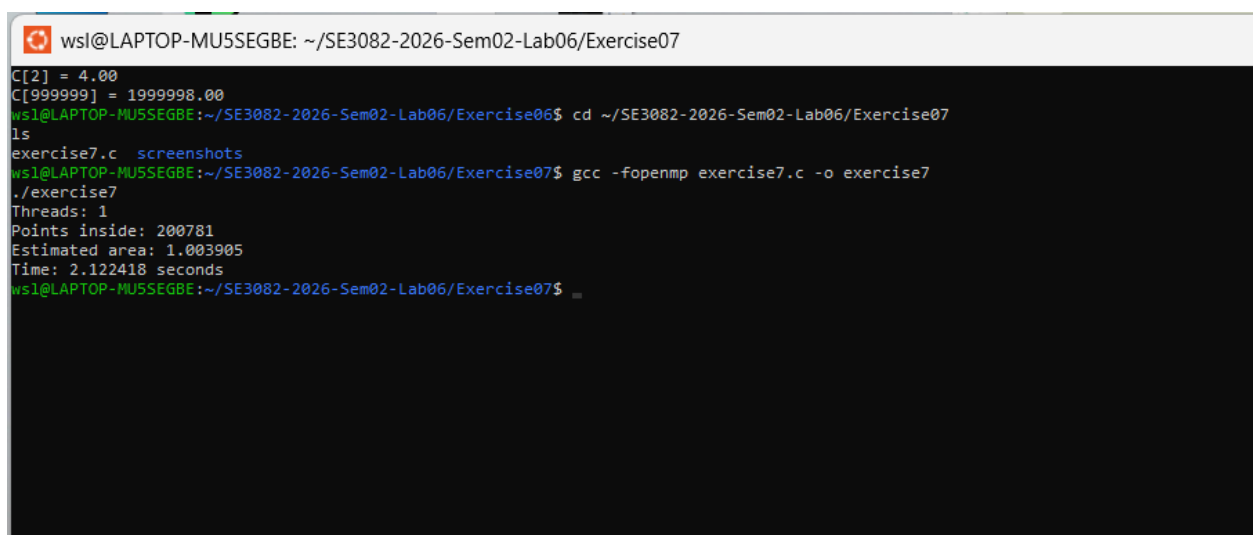
The arrays are divided into fixed-size strips containing 32 elements using strip mining. OpenMP parallelizes the processing of these strips across multiple threads. Within each strip, omp simd enables SIMD vectorization of the element-wise multiplication. The strip size of 32 provides a suitable block size for efficient vector processing and parallel execution.

```
wsl@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise06
r = 4, k = 9, thread # = 3
r = 4, k = 10, thread # = 3
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise04$ cd ~/SE3082-2026-Sem02-Lab06/Exercise05
ls
exercise05.txt  exercise5.c  screenshots
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise05$ gcc -fopenmp exercise5.c -o exercise5
./exercise5
Fibonacci(10) = 55
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise05$ cd ~/SE3082-2026-Sem02-Lab06/Exercise06
ls
exercise6.c  screenshots
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise06$ gcc -fopenmp exercise6.c -o exercise6
./exercise6
C[0] = 0.00
C[1] = 2.00
C[2] = 4.00
C[999999] = 1999998.00
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise06$
```

Exercise 07

The outer loop was parallelized using the `#pragma omp parallel for` directive. The iterations of the outer loop are distributed among the available threads. The inside variable is declared as a reduction variable so that each thread maintains its own local count, and the results are combined at the end of the parallel region.

The `omp_get_wtime()` library routine is used to measure the execution time. The program was executed using 1, 2, 3, and 4 threads. The calculated Mandelbrot area should remain identical for all thread counts, while the execution time may vary depending on the number of threads used.



```
wsl@LAPTOP-MU5SEGBE: ~/SE3082-2026-Sem02-Lab06/Exercise07
C[2] = 4.00
C[9999999] = 1999998.00
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise06$ cd ~/SE3082-2026-Sem02-Lab06/Exercise07
ls
exercise7.c  screenshots
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise07$ gcc -fopenmp exercise7.c -o exercise7
./exercise7
Threads: 1
Points inside: 200781
Estimated area: 1.003905
Time: 2.122418 seconds
wsl@LAPTOP-MU5SEGBE:~/SE3082-2026-Sem02-Lab06/Exercise07$
```